Final Assignment - Object Grouping by Shape

In22-S7-CS4373 - Machine Vision

Submitted by: Gunasekara T I U (220192J)
Submitted date: 28/10/2026
GitHub Link: [CS4373_machine_vision/Final_assignment]

Table of Content

Task **2**
Sample Images **2**
Proposed Pipeline **3**
Key Steps **4**
Preprocessing 4
1. Median Filter (Outlier Noise) 4
2. Gaussian Filter (Gaussian Noise) 4
Segmentation 5
1. Histogram Smoothing 5
2. Threshold Selection: Valley Point 5
Polarity and Binary Clean-up 6
Boundary Chain Code 7
1. Contour Tracking 7
2. 32-Digit Chain Code 7
RST and Starting-Point Invariance 8
1. For Rotation invariance - the first difference of the chain code: 8
2. For Starting point invariance - redefine the starting point to form the minimum magnitude sequence 8
3. Orientation from Image Moments 9
4. Start Points and Offsets 9
Invariance Summary 10
Grouping 10
Results **12**
Limitations **12**

Task

Develop a machine-vision pipeline that:

1. Loads `.png` and `.jpg` images from a folder specified by a command-line argument. Each image contains one object with foreground and background noise.
2. Processes each image by:
    a. Reducing Gaussian and outlier noise.
    b. Segmenting the object from the background through thresholding.
    c. Generating a 32-digit chain code representing the object boundary.
    d. Making the chain code invariant to rotation, scaling, translation (RST), and starting-point selection.
3. Group images with similar objects despite RST variations based on the chain code.
4. Save the output (i.e. grouped image file names) into a text file.

Run the pipeline using the sample images included in the file named “samples.zip”

Sample Images

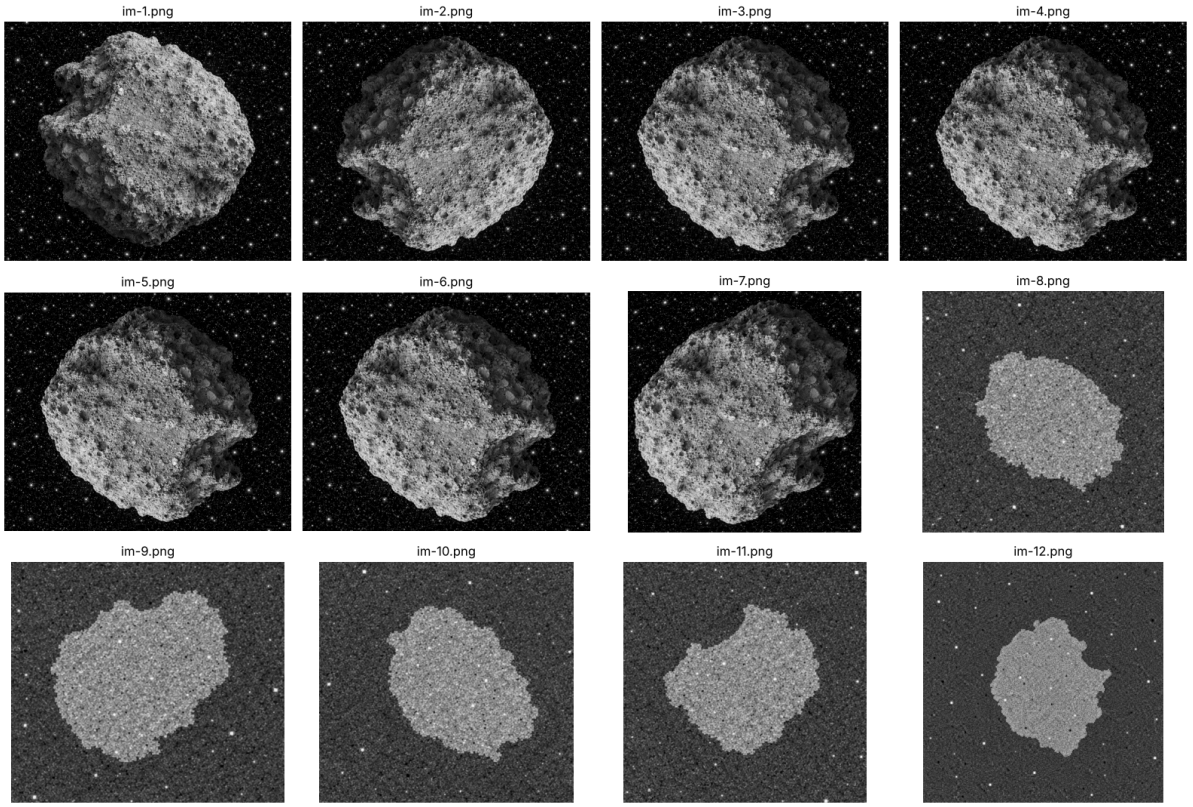


Figure 1: Sample Images

Proposed Pipeline

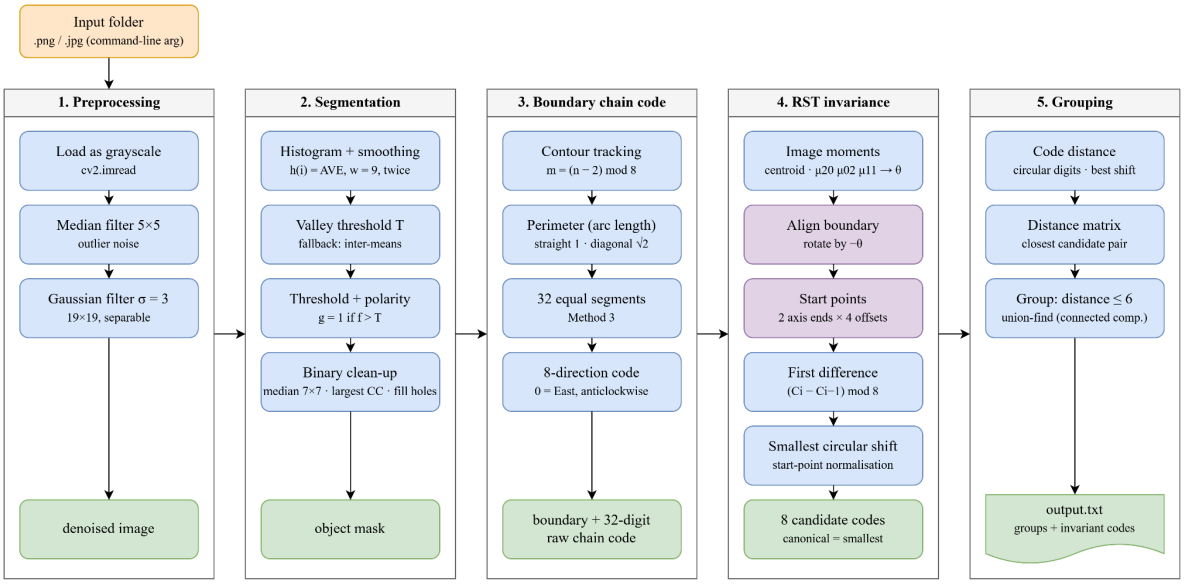


Figure 2: Overview of the proposed pipeline

The pipeline runs five stages on every image, then compares all images with each other:

1. **Image loading and Preprocessing (noise reduction):** a median filter removes outliers, then a Gaussian filter removes Gaussian noise.
2. **Segmentation:** a threshold taken from the histogram valley separates the object; binary clean-up leaves one solid object region.
3. **Boundary chain code:** contour tracking gives the ordered boundary, which is cut into 32 equal-length segments and coded with 8 directions (scaling and translation invariance are achieved).
4. **RST and start-point invariance:** the boundary is aligned to its major axis, then the first difference (for rotation invariance) and the smallest circular shift (for start point invariance) are applied.
5. **Grouping:** images whose invariant codes are within a distance threshold are linked, and the linked sets are written to output.txt.

Stage	Input	Output
1. Noise reduction	image file	denoised grey image
2. Segmentation	denoised image	binary object mask (object = 1)
3. Boundary chain code	object mask	ordered boundary pixels, raw 32-digit code
4. RST invariance	mask and boundary	8 invariant candidate codes per image
5. Grouping	candidate codes of all images	groups of file names (output.txt)

Note:
The pipeline is run with:
`python pipeline.py Samples [--output output.txt] [--threshold 6] [--debug].`

With `--debug` it saves the intermediate figures used in this report to a debug folder.

Key Steps

Preprocessing

Images are loaded as 8-bit grayscale. Two filters are applied, in this order.

1. **Median Filter (Outlier Noise)**
   - ➢ Each pixel is replaced by the median of its 5x5 neighbourhood.
   - ➢ Salt-and-pepper noise and small bright stars are are removed.
   - ➢ The image is padded by replicating its edge pixels so that the window fits at the borders.

2. **Gaussian Filter (Gaussian Noise)**
   - ➢ The 2-D Gaussian is separable, so it is applied as a horizontal 1-D pass followed by a vertical 1-D pass, which needs 2k instead of k² multiplications per pixel.
   - ➢ The median filter runs first so that outliers are removed before they can be spread by averaging.
   - ➢ Since for object segmentation only the silhouette is sufficient, a strong smoothing with k = 19 is selected, so the surface texture will be smothen.

```
REDUCE_NOISE(f):
    g ← MEDIAN_FILTER(f, 5)                    // remove outliers first
    k ← 2·⌈3σ⌉ + 1                             // σ = 3 → k = 19
    w(x) ← exp(-x² / 2σ²) / Σ exp(-x² / 2σ²),  x = -k/2 … k/2
    g ← CONVOLVE_ROWS(g, w)                    // separable Gaussian:
    g ← CONVOLVE_COLUMNS(g, w)                 // rows, then columns
    return g
```

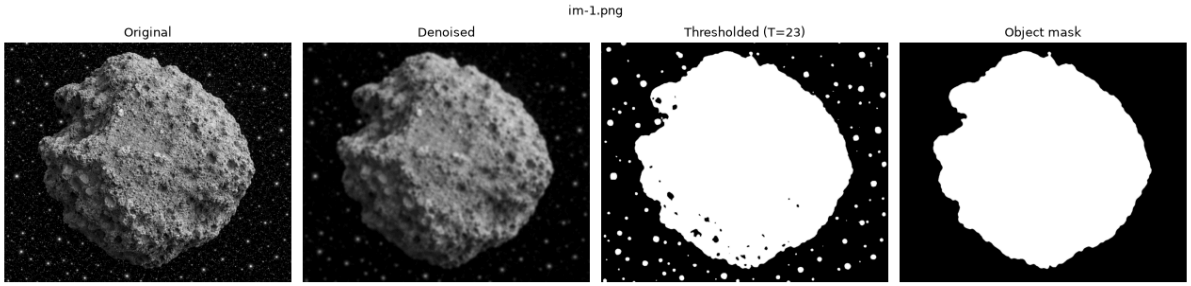


Figure 3: Original, denoised, thresholded and final mask for one image (im-1.png)

Segmentation

The object is separated from the background with a global threshold T chosen from the grey-level histogram.

1. Histogram Smoothing

Each intensity bin is replaced by the average of its neighbours (9) to remove the small false valleys and peaks.

$$h(i) = AVERAGE(h(i-4) \dots h(i+4)); \quad i = 9$$

The smoothing is applied twice.

2. Threshold Selection: Valley Point

- ➢ Since the background covers most of the image, the largest peak of the smoothed histogram is the background peak.
- ➢ The search starts at the highest peak and moves one grey level at a time towards the side of the peak that holds more pixels, which is where the object's pixels are. It stops when the histogram starts rising again, which is the valley - T.
- ➢ If the search reaches the end of the grey scale without finding a valley, the histogram is not bimodal and the inter-means algorithm is used instead.

```
VALLEY_THRESHOLD(h):
    p ← argmax h                                  // background peak
    step ← +1 if Σ h[p+1 … 255] > Σ h[0 … p−1] else −1
    v ← p
    while 0 < v + step < 255 and h[v + step] ≤ h[v]:
        v ← v + step                              // still downhill
    if v + step reached 0 or 255:
        return INTER_MEANS(h)                     // no valley found
    return v

INTER_MEANS(h):
    T ← mean grey level of the image
    repeat
        μ1 ← mean of levels ≤ T;   μ2 ← mean of levels > T
        T ← (μ1 + μ2) / 2
    until μ1 and μ2 change by less than 0.5
    return T
```

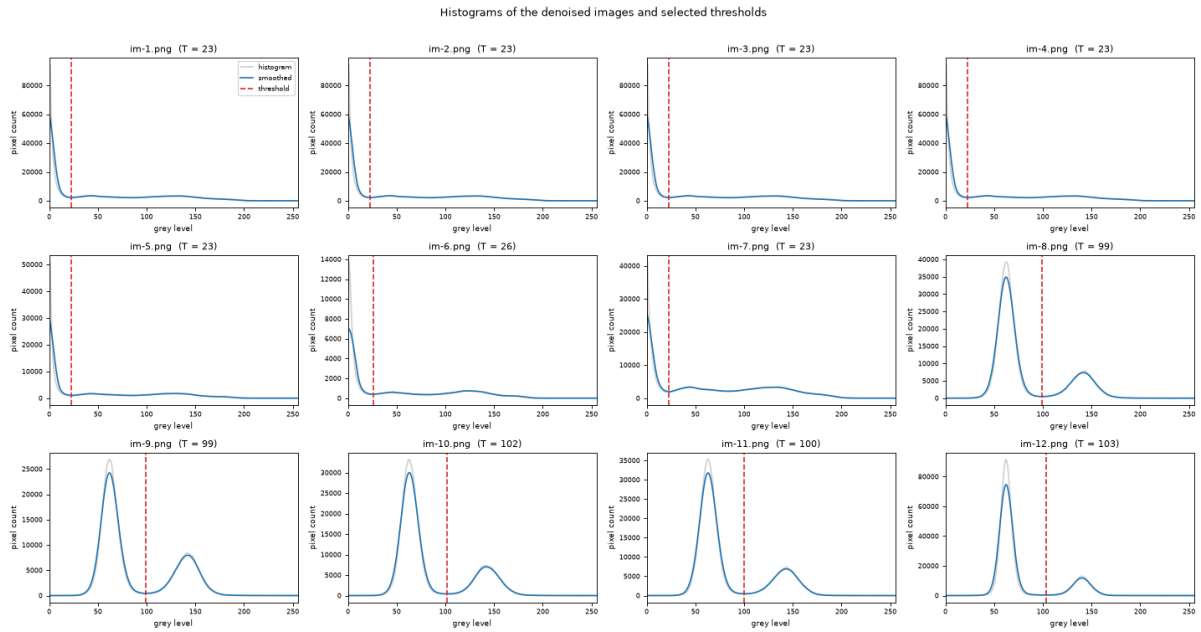


Figure 4: Histograms of all images with the selected threshold (debug/histograms.png)

Polarity and Binary Clean-up

The binary image:

$$g(x, y) = 1 \; if \; f(x, y) > T, \; otherwise \; 0$$

If more than half of the image border is 1, the object must be darker than the background, and the image is inverted so that the object is always 1.

The thresholded image still contains stars and small holes, so it is cleaned in three steps:

- **Median filter 7x7 on the binary image:** removes isolated specks and smooths ragged edges, and the result stays binary.
- **Largest connected component (1 → 0):** every object pixel is labelled by 8-connected region growing, and only the largest region is kept.
- **Hole filling (0 → 1):** the background is grown from the image border with 4-connectivity. Background pixels that cannot be reached are holes inside the object and are set to 1. 4-connectivity is used for the background because the object uses 8-connectivity, so the background cannot leak through diagonal gaps.

```
LARGEST_COMPONENT(b):                              // region growing
    label ← 0 for every pixel;  n ← 0
    for each pixel s with b[s] = 1 and label[s] = 0:
        n ← n + 1;  label[s] ← n;  stack ← [s]
        while stack not empty:
            q ← pop(stack)
            for each 8-neighbour r of q:
                if b[r] = 1 and label[r] = 0:
                    label[r] ← n;  push(stack, r)
    return pixels whose label occurs most often

FILL_HOLES(b):
    outside ← background pixels on the image border
    grow outside through 4-neighbours r with b[r] = 0
    return every pixel not in outside              // object + holes
```

Boundary Chain Code

1. Contour Tracking

The outer boundary of the mask is traced with the contour-tracking algorithm. The neighbours of a pixel are numbered 0 to 7 clockwise, starting at the top-right.

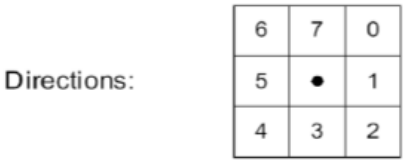


Figure 5: Contour tracking direction order

```
TRACE_CONTOUR(mask):
    start ← first object pixel when scanning from the top-left corner
    boundary ← [start];  current ← start;  n ← 1
    loop
        m ← (n − 2) mod 8
        while neighbour of current in direction m is background:
            m ← (m + 1) mod 8
        next ← neighbour of current in direction m
        if next = start: stop
        append next to boundary;  current ← next;  n ← m
    return boundary
```

2. 32-Digit Chain Code

- ➢ A chain code with one digit per boundary pixel (Method 1) is long and sensitive to noise.
- ➢ The pipeline uses Method 3: the boundary is divided into 32 segments of equal length, and the straight line joining the ends of each segment is coded with the 8-direction code (0 = East, then anticlockwise in 45° steps).
- ➢ Lengths along the boundary use the perimeter rule: straight step → 1, diagonal step → √2.
- ➢ (Counting pixels instead would make segments along diagonal edges physically longer and would change the cut points when the object is rotated).

```
CHAIN_CODE(boundary, start_length = 0):
    for each step between neighbouring boundary pixels:
        length ← 1 if the step is straight, √2 if diagonal
    arc ← running sum of the lengths;  P ← total perimeter
    for k = 0 … 32:
        cut[k] ← first pixel with arc ≥ (start_length + k·P/32) mod P
    for k = 0 … 31:
        (Δrow, Δcol) ← cut[k+1] − cut[k]
        code[k] ← round(atan2(−Δrow, Δcol) / 45°) mod 8
    return code
```

This code is already invariant to translation, because it stores only directions, and to scale, because every boundary is cut into the same number of segments. But it is not invariant to rotation or to the starting point.

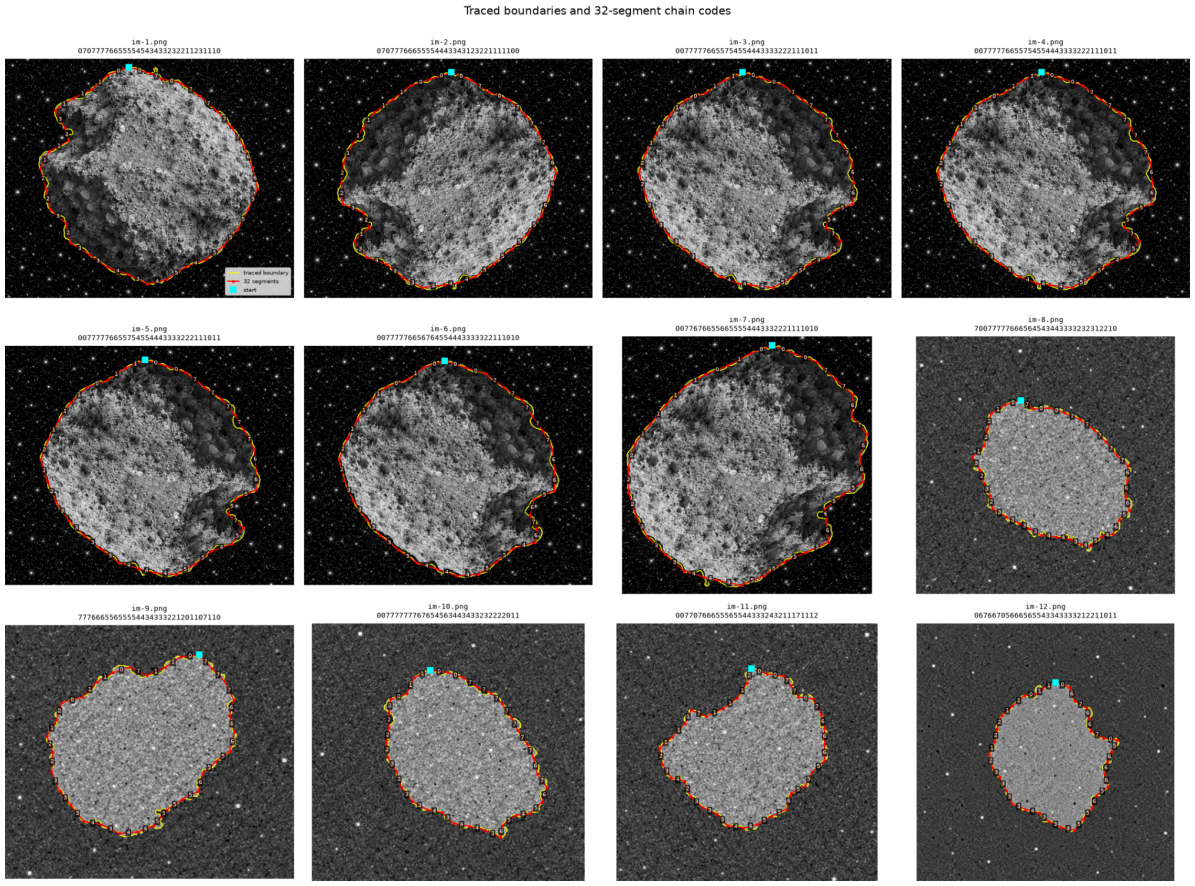


Figure 6: Traced boundaries, 32 segments and raw chain codes (debug/chain_codes.png)

RST and Starting-Point Invariance

1. For Rotation invariance - the first difference of the chain code:

$$C'_i = (C_i - C_{i-1}) \; mod \; 8 \text{ (code treated as circular)}$$

Reason: Rotating the shape by a multiple of 45° adds the same constant to every digit.

2. For Starting point invariance - redefine the starting point to form the minimum magnitude sequence.

example:

1. code	2 1 7 0 7 5 5 2
2. First difference	0 7 6 1 7 6 0 0 5
3. Smallest circular shift	0 0 5 0 7 6 1 7 6

The normalisations are exact only if the boundary itself is unchanged by rotation, which is not true for a discrete image. Two effects were responsible.

1. The 32 cuts start at the top-most pixel, which is a different part of the shape after rotation, so the chords themselves change.

2. Chord directions are rounded to the nearest 45°, so a small rotation can change a digit.

Solution: puts every boundary into a standard pose before the two normalisations.

3. Orientation from Image Moments

The moments of the mask give the centroid and the orientation of the shape. With

$$m_{pq} = \sum_x \sum_y x^p y^q \text{ over the object pixels,}$$

the centroid:

$$(\bar{x}, \bar{y}) = (\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}})$$

the second-order central moments:

$$\mu_{11} = m_{11} - \bar{y} \cdot m_{10}, \mu_{20} = m_{20} - \bar{x} \cdot m_{10}, \mu_{02} = m_{02} - \bar{y} \cdot m_{01}.$$

The angle of the major axis:

$$\theta = \frac{1}{2} \cdot atan2(2\mu_{11}, \mu_{20} - \mu_{02})$$

Every boundary point is moved so that the centroid is at the origin and rotated by −θ, which lays the major axis along the x-axis..

4. Start Points and Offsets

- ➢ The cuts start at a boundary pixel at an end of the major axis (largest or smallest x after alignment). This point belongs to the shape, so it moves with it when the object is rotated.
- ➢ The axis has no direction, so both ends are used.
- ➢ Noise can move the end pixel by one or two pixels, which would move all 32 cuts, so the cuts are also tried at offsets of ¼, ½ and ¾ of a segment.
- ➢ This gives 2 × 4 = 8 candidate codes per image. The cut positions are measured along the original boundary, whose length does not change with rotation, and the directions are measured on the aligned boundary.

```
INVARIANT_CODES(mask, boundary):
    (x̄, ȳ, μ20, μ02, μ11) ← CENTRAL_MOMENTS(mask)
    θ ← ½ · atan2(2μ11, μ20 − μ02)
    B′ ← rotate (boundary − (x̄, ȳ)) by −θ            // standard pose
    L ← perimeter / 32;  candidates ← {}
    for start in {argmax x′(B′), argmin x′(B′)}:      // both axis ends
        for p in {0, 1, 2, 3}:                        // sub-segment offsets
            C ← CHAIN_CODE on B′ from arc(start) + p·L/4
            D ← [(C[i] − C[i−1]) mod 8 for each i]    // first difference
            candidates ← candidates ∪ {MIN_CIRCULAR_SHIFT(D)}
    return candidates            // printed code = smallest candidate
```

Invariance Summary

Invariance	**Achieved by**
Translation	directions only; moments taken about the centroid
Scale	32 equal-length segments
Rotation	major-axis alignment, then first difference
Starting point	axis-end start with offsets, then smallest circular shift

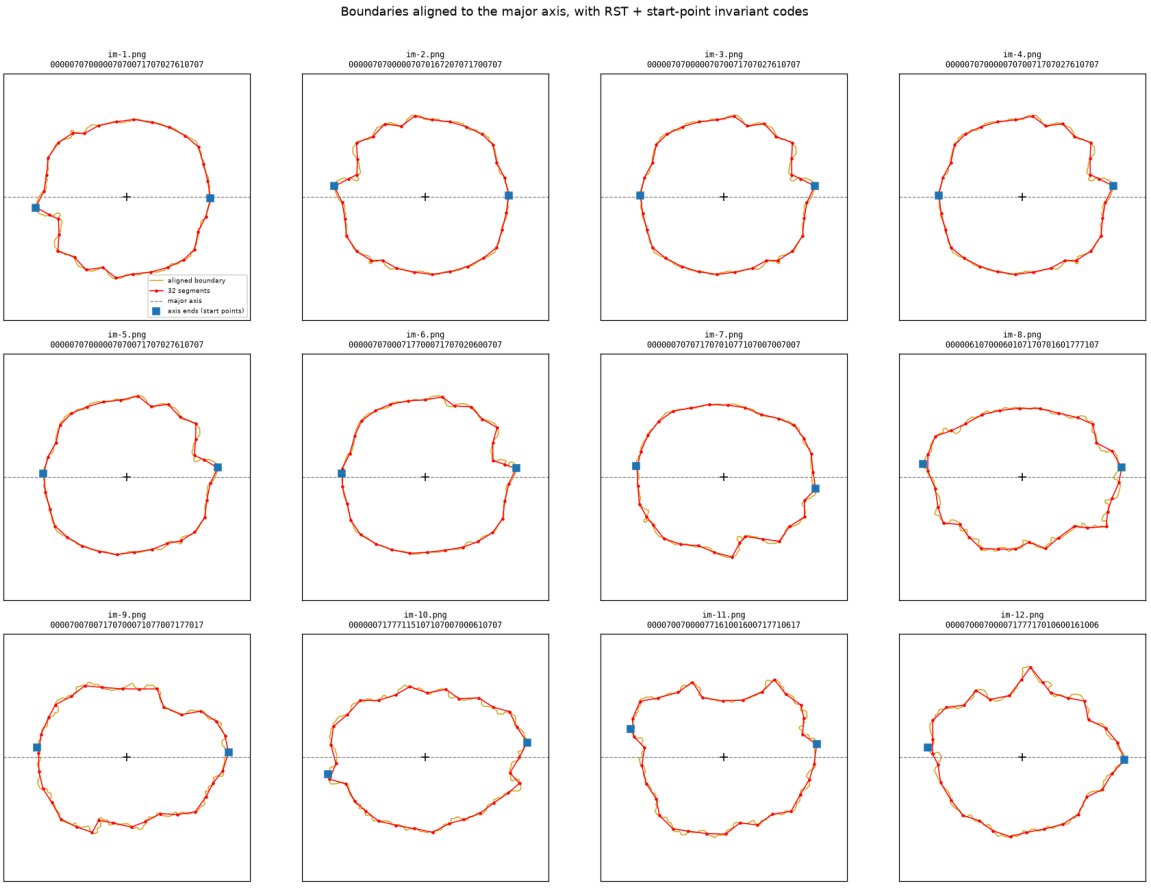


Figure 7: Boundaries aligned to the major axis, with start points and invariant codes (debug/aligned.png)

Grouping

➢ Two code digits are compared circularly:

$$d(a,\ b)\ =\ min(|a\ -\ b|,\ 8\ -\ |a\ -\ b|)$$

so that 0 and 7 (no turn and a 45° right turn) are one step apart.

➢ The distance between two codes is the sum of the digit differences at the best of the 32 circular alignments.

➢ The distance between two images is the smallest distance between any of their candidate codes.

➢ Two images are linked when their distance is at most a threshold, and the groups are the connected components of these links, found with a union-find structure (same idea as connected component labelling, applied to images instead of pixels).

```
GROUP(images, threshold):
    for each pair (i, j):
        D[i, j] ← min over candidates a of i, b of j of
                  min over shifts s of Σ_k d(a[k], b[(k + s) mod 32])
    parent[i] ← i for every image
    for each pair (i, j) with D[i, j] ≤ threshold:
        parent[ROOT(j)] ← ROOT(i)                    // merge the two groups
    return images grouped by ROOT
```

The threshold was chosen from the distance matrix:

Threshold value	Outcome
4 - 8	Gives same groups
< 4	Rotated copies are split away
10 <	Unrelated shapes are merged

Therefore, the default threshold = 6

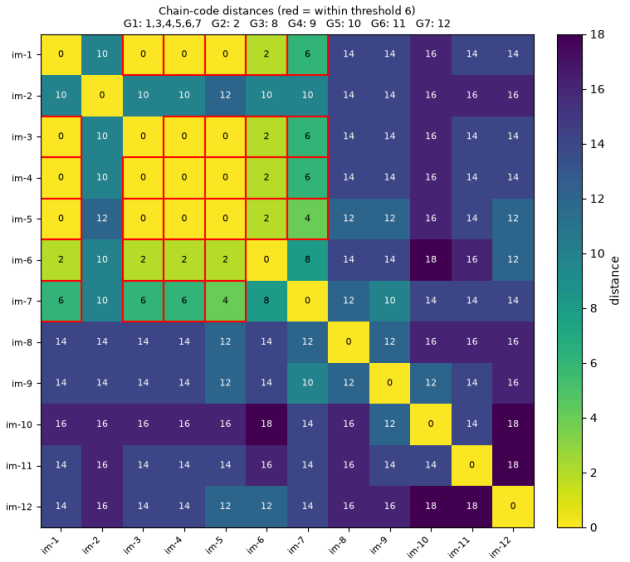


Figure 8: Chain-code distances between all images; red cells are within the threshold (debug/distance_matrix.png)

Results

Running the pipeline on the 12 sample images writes the following groups to output.txt:

Group	Images	Invariant code (first image)
1	im-1, im-3, im-4, im-5, im-6, im-7	00000707000007070071707027610707
2	im-2	00000707000007070167207071700707
3	im-8	00000610700060107170701601777107
4	im-9	00007007007170700071077007177017
5	im-10	00000071777115107107007000610707
6	im-11	00007007000077161001600717710617
7	im-12	00007000700007177717010600161006

Limitations

- The valley search assumes the object's pixels lie on the side of the background peak that holds more pixels, and the polarity check assumes the object covers less than half of the image border.
- Mirror images (im-2) are not matched, and non-uniform stretching (im-7) is outside RST; it groups correctly here only because the stretch is small.
- Grouping uses single linkage, so an image can join a group through an intermediate image: im-6 and im-7 are 8 apart, but both are close to im-5.